

GPU Execution Optimization: Placement and Interview Guide

This guide assumes NVIDIA-style terms such as a **warp** (usually 32 threads), but the ideas also apply to AMD wavefronts and other SIMT hardware. Exact limits—warp size, register-file size, maximum resident blocks, and instruction rates—depend on the GPU architecture, so verify device specifications when tuning real code.

Warp Divergence

1. Overview

Warp divergence occurs when threads in the same warp take different control-flow paths. A GPU issues one instruction to a group of lanes; if some lanes need the **if** path and others need the **else** path, the warp normally executes both paths with different lanes masked off. Divergence matters because inactive lanes consume issue time without doing useful work. It appears in image filters, graph traversal, ray tracing, parsing, and any data-dependent kernel. Interviewers ask about it to test whether you understand SIMT execution rather than treating a GPU as many unrelated CPU cores.

2. Core Idea

Think of a warp as 32 students following one instructor. If half must solve problem A and half problem B, the instructor teaches A while the B group waits, then teaches B while the A group waits.

```
if (x[tid] >= 0) y[tid] = sqrtf(x[tid]);
else           y[tid] = 0.0f;
```

Step by step: threads load different values; the predicate is evaluated per lane; the warp executes the true path for enabled lanes; it executes the false path for the remaining lanes; the paths reconverge. If every lane makes the same choice, there is no divergence even though a branch exists.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
SIMT execution	One instruction is issued across multiple lanes.	32 additions execute together.	Contrast SIMT with independent CPU threads.
Active mask	Bit mask identifies lanes participating in an instruction.	16 lanes enabled on an if path.	Explain why masked lanes waste capacity.
Reconvergence	Split paths eventually join at a compiler/hardware-selected point.	Both paths meet after an if .	Divergence is usually local, not permanent.

Subtopic	Meaning and importance	Example	Interview angle
Nested divergence	Nested conditionals can create smaller active groups.	Material type, then light type.	Discuss rapidly falling warp efficiency.
Data reordering	Group similar inputs so adjacent threads choose alike.	Sort rays by material.	Optimization can cost preprocessing and memory traffic.
Predication	Short branches may become conditional instructions rather than explicit jumps.	Conditional assignment.	Predication still executes instructions for lanes whose result is discarded.

4. Real-World Example

In a ray tracer, rays hit different materials. If one warp contains diffuse, glass, and metal rays, each shading path runs with only some lanes active. A common optimization compacts or sorts rays by material before shading. It adds queue-management work, but coherent warps can repay that cost when shaders are long.

5. Diagrams / Mental Models

```

Warp lanes:  0 1 2 3 4 5 6 7
condition:   T T F T F F T F
IF path:     A A - A - - A -
ELSE path:   - - B - B B - B
rejoin:      C C C C C C C C

```

Useful mental model: approximate branch efficiency as useful active-lane work divided by total lane slots issued across all paths. It is not simply “number of branches.”

6. Common Interview Questions

- What is warp divergence?** Threads in one warp follow different paths, forcing serialized or masked execution. Expected: same-warp scope and lost lane utilization. Mistake: saying different warps must follow the same path.
- Does every `if` cause divergence?** No; only differing decisions within a warp do. Expected: warp-uniform conditions. Mistake: recommending removal of all branches.
- What happens to the nonparticipating threads?** Their lanes are masked while the other path executes. Expected: they wait logically but still occupy the warp. Mistake: claiming they run another warp’s instruction independently.
- Is divergence a correctness problem?** Normally no; it is primarily performance-related. Expected: synchronization assumptions can create correctness hazards. Mistake: treating different outputs as incorrect.
- Can divergence occur across blocks?** Blocks and warps are scheduled independently; the relevant penalty is within a warp. Mistake: measuring branch balance over the whole grid.
- How can data layout reduce divergence?** Arrange similar items next to each other so neighboring thread IDs make the same decisions. Mistake: sorting without considering its cost.

7. **Does replacing a branch with arithmetic always help?** No; extra instructions may cost more than a coherent branch. Expected: benchmark and consider compiler predication. Mistake: branchless code as a universal rule.
8. **How do loops diverge?** A warp continues until its slowest lane exits; completed lanes become inactive. Mistake: using average iteration count as warp cost.
9. **How is divergence measured?** Use profiler metrics for branch efficiency, active threads per warp, and source-level stalls. Mistake: inferring it only from runtime.
10. **Why are boundary checks often cheap?** Only edge warps diverge, so the affected fraction can be small. Mistake: assuming one divergent warp ruins the whole kernel.

7. Deep-Dive Questions

1. **How does independent thread scheduling change divergence?** Newer GPUs track execution state more flexibly per thread, but lanes sharing issue resources still benefit from coherent execution; it does not make divergence free.
2. **Can warp-level primitives be used inside divergent code?** Yes only with a correct participation mask and convergence assumptions. Passing a mask containing lanes that never reach the primitive can hang or produce invalid results.
3. **When is compaction worth it?** When saved inactive-lane work exceeds scan, scatter, queue, and lost-locality costs; long heterogeneous workloads benefit most.
4. **Why can a branch outperform predication?** A branch skips a long path for all lanes when the condition is uniform, while predication may issue every instruction.
5. **How do memory accesses interact with divergence?** Fewer active lanes may generate sparse, poorly coalesced transactions, combining control-flow and bandwidth inefficiency.

8. Comparison Tables

Coherent warp	Divergent warp
All lanes choose one path	Lanes choose multiple paths
High lane utilization	Some lane slots are masked
Cost roughly chosen path	Cost approaches sum of taken paths
Common with regular arrays	Common with irregular graphs/rays

9. Common Mistakes

- Confusing a branch with divergence.
- Optimizing divergence before profiling its frequency and path cost.
- Grouping data by branch outcome while destroying memory locality.
- Assuming divergence between separate warps is harmful.
- Using warp intrinsics with an incorrect active mask.

10. Edge Cases / Special Cases

Short branches may be predicated; uniform branches can be broadcast efficiently; a single edge warp may have negligible global impact; loop divergence is governed by the maximum trip count in each warp; architecture-specific scheduling changes implementation details but not the value of coherent lanes.

11. How to Explain in Interview

“Warp divergence happens when threads in the same SIMT warp take different paths. The GPU executes the required paths with lane masks, so inactive lanes waste issue slots. I reduce it by making work warp-coherent, but only when profiling shows the saved work exceeds reordering or extra-instruction costs.”

12. Quick Revision Notes

- Scope: one warp, not the entire grid.
- Branch present does not imply divergence.
- Cost depends on path length and active-lane distribution.
- Loops wait for the slowest lane.
- Interview trap: branchless is not automatically faster.

13. Practice Tasks

1. Write a CUDA kernel whose condition is `(threadIdx.x & 1)` and another whose condition is `(threadIdx.x / warpSize) & 1`; predict which diverges.
2. Profile both kernels and compare branch and warp-execution efficiency.
3. Implement a variable-length loop and plot runtime against the maximum per-warp length.
4. Reorder a set of classified records by class, then measure whether coherent processing pays for the reorder.

14. Final Cheat Sheet

Item	Remember
Core definition	Same warp, different paths
Why it matters	Masked lanes reduce useful throughput
Most asked	Does every branch diverge? No
Main comparison	Coherent path vs serialized masked paths
One-line answer	Divergence turns lane-level parallelism into partially idle execution.

Branch Divergence

1. Overview

Branch divergence is the control-flow situation in which threads grouped for SIMT execution evaluate a branch differently. It is the common cause of warp divergence. The phrase is often used interchangeably with warp divergence, but a precise interview answer distinguishes the cause (different branch outcomes) from the execution effect (a warp running paths with masks). It matters in conditional algorithms such as thresholding, sparse processing, collision detection, and graph analytics.

2. Core Idea

A highway toll plaza has one lane assigned to a busload of passengers: if some need cash processing and others use a pass, the group cannot finish as one uniform transaction. On a GPU, the key question is not whether code contains `if`, but whether the condition varies at the hardware group's granularity.

```
// Usually divergent: adjacent lanes alternate.
if (threadIdx.x % 2) odd_work(); else even_work();

// Uniform within each warp when block layout aligns with warpSize.
if (threadIdx.x / warpSize) odd_work(); else even_work();
```

The compiler generates control flow or predicates; each lane evaluates the condition; hardware forms masks; paths execute; execution reconverges.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Uniform condition	Same result for all lanes in a warp.	<code>if (blockIdx.x == 0)</code>	A branch can be cheap even when blocks differ.
Data-dependent branch	Outcome depends on each element.	<code>if (value > threshold)</code>	Input distribution controls performance.
Branch granularity	Pattern relative to warp lane grouping.	Alternating IDs vs groups of 32.	Global 50/50 balance says little.
Predication	Conditional writes controlled by predicates.	<code>out = p ? a : b.</code>	Suitable for short paths, not free.
Branch elimination	Reformulate with min/max or arithmetic.	Clamp using <code>fminf/fmaxf</code> .	Extra work and numerical behavior matter.
Work partitioning	Create separate queues/kernels by type.	Process valid and invalid records separately.	Kernel-launch and partition costs matter.

4. Real-World Example

An image segmentation kernel labels pixels above a threshold. Natural images have coherent regions, so interior warps often agree while boundary warps diverge. Storing pixels spatially preserves this coherence. Randomly shuffling pixels could retain coalescing poorly and make nearly every warp mixed.

5. Diagrams / Mental Models

```
Same global split, different local behavior:
Grouped:    T T T T | F F F F -> two coherent warps
Alternating: T F T F | T F T F -> two divergent warps
```

Judge branches per warp using: outcome coherence × path cost × frequency.

6. Common Interview Questions

1. **Define branch divergence.** Lanes in one warp evaluate a control branch differently. Expected: granularity. Mistake: calling any `if` divergent.
2. **How is it related to warp divergence?** It is a control-flow cause of the masked multi-path warp execution. Mistake: inventing unrelated concepts.
3. **Can a 50/50 branch be efficient?** Yes, if complete warps choose different paths. Mistake: looking only at global probability.
4. **What branch pattern is worst?** Long paths taken by complementary subsets within most warps. Mistake: saying exactly 50/50 without path context.
5. **Can the compiler remove divergence?** It may predicate, simplify, or prove uniformity, but data-dependent outcomes remain work. Mistake: assuming optimization erases path cost.
6. **When should you split a kernel?** When paths are large and stable grouping pays for partition and launch overhead. Mistake: splitting tiny branches automatically.
7. **Why may `?:` not solve it?** Syntax does not determine generated instructions; both values may still be computed. Mistake: equating ternary with branchless speed.
8. **How do you diagnose it?** Profile branch efficiency and active lanes, then inspect source/SASS correlation. Mistake: rewriting based only on source appearance.
9. **Do switch statements diverge?** Yes when lanes choose different cases; total cost can approach all cases represented. Mistake: treating switch as special.
10. **Can branch divergence improve memory behavior?** Occasionally a branch avoids unnecessary loads, so removing it can increase traffic. Expected: total-cost reasoning. Mistake: optimizing one metric alone.

7. Deep-Dive Questions

1. **What is a uniform branch optimization?** The compiler/hardware recognizes a warp-wide decision and transfers control once without splitting lane execution.
2. **How do early returns behave?** Returned lanes become inactive while remaining lanes continue; highly variable work lengths reduce utilization.
3. **Can atomics inside one path amplify the issue?** Yes; participating lanes may serialize at the atomic while other lanes remain inactive, stacking bottlenecks.
4. **How does function inlining affect analysis?** It exposes branches to optimization but may increase code size and register pressure; measure the whole kernel.
5. **Why can sorting hurt despite reducing branches?** Sorting consumes bandwidth, adds latency, and may break spatial locality or output ordering.

8. Comparison Tables

Branch divergence	Warp divergence
Different outcomes at a branch	Execution consequence across paths
Source/control-flow view	Hardware/SIMT utilization view
Avoided by warp-uniform decisions	Also arises from variable loop/return behavior
Explicit branch	Predication
Can skip long uniform path	Usually issues short guarded operations

Explicit branch**Predication**

Diverges when outcomes differ

Avoids control transfer, not instruction work

9. Common Mistakes

- Treating global branch balance as the metric.
- Replacing a cheap branch with expensive unconditional math.
- Ignoring loop exits and early returns as divergent control flow.
- Assuming source syntax maps directly to machine branching.
- Separating work without including partition overhead in benchmarks.

10. Edge Cases / Special Cases

Warp-uniform conditions are not divergent; boundary branches affect only some warps; compiler predication thresholds vary; both sides can still cause different memory transactions; numerical replacements such as `abs`, `min`, or arithmetic masks may change NaN, overflow, or signed-zero behavior.

11. How to Explain in Interview

“Branch divergence means lanes in one warp disagree on a branch. Hardware must service the represented paths under different active masks. I reason at warp granularity, check path length and input coherence, and consider predication or work grouping only after profiling.”

12. Quick Revision Notes

- Cause: differing per-lane branch outcomes.
- Effect: masked path execution and lower lane utilization.
- Uniform per warp is efficient even if warps differ.
- Predication trades control flow for unconditional instruction issue.
- Trap: syntactically branchless does not mean less work.

13. Practice Tasks

1. Compare alternating, random, and warp-grouped predicates with identical global true counts.
2. Inspect compiler output for a two-instruction conditional and a long conditional.
3. Benchmark one mixed kernel against partition-plus-two-kernels.
4. Test arithmetic branch replacement on NaN and extreme inputs.

14. Final Cheat Sheet

Item	Remember
Core	Same-warp lanes disagree at control flow
Matters	Multiple paths consume issue time
Asked	Branch divergence vs warp divergence
Compare	Uniform branch, divergent branch, predication

Item	Remember
One line	Measure branch coherence per warp, not branch frequency globally.

Occupancy

1. Overview

Occupancy is the ratio of active warps resident on a streaming multiprocessor (SM) to the architectural maximum active warps on that SM. Resident blocks consume finite registers, shared memory, warp slots, and block slots. Occupancy matters because more ready warps can hide latency, but maximum occupancy is neither necessary nor sufficient for maximum performance. It is used when tuning virtually every CUDA/HIP kernel. Interviewers ask it to expose the common misconception that occupancy equals utilization or speed.

2. Core Idea

Imagine a restaurant kitchen. Occupancy is how many orders are present relative to capacity; utilization is whether cooks are actually busy. More orders help when one waits for an oven, but overcrowding may force each dish to use fewer tools.

Step by step: choose a block size; compiler assigns registers per thread; kernel requests shared memory per block; hardware determines how many whole blocks fit on one SM; those blocks contribute resident warps; divide by the maximum supported warps. For example, if 32 warps are resident and the SM limit is 64, theoretical occupancy is 50%.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Theoretical occupancy	Limit predicted from resources and launch shape.	Calculator reports 50%.	It is a capacity estimate, not runtime activity.
Achieved occupancy	Profiler-observed average resident warps.	Tail waves reduce average.	Explain why it differs from theoretical.
Register limit	Registers per thread $\times$ threads restrict blocks.	96 registers/thread permits fewer blocks.	Register capping may spill.
Shared-memory limit	Per-block allocation restricts resident blocks.	48 KiB block on 100 KiB SM.	Dynamic shared memory is included.
Granularity	Allocation happens in hardware-sized units and whole blocks.	A slight register rise drops one block.	Occupancy changes in steps.
Sufficient occupancy	Enough ready warps to cover relevant stalls.	Compute-heavy ILP kernel at 50%.	Chasing 100% can reduce performance.

4. Real-World Example

A tiled matrix multiplication uses shared memory and registers to reuse data. Larger tiles may lower occupancy, yet reduce global-memory traffic and increase arithmetic work per load. High-performance GEMM therefore often chooses more reuse and instruction-level parallelism over maximum resident warps.

5. Diagrams / Mental Models

```
blocks per SM = min(
    thread/warp limit,
    register-file limit,
    shared-memory limit,
    architectural block limit)

occupancy = resident warps / maximum warps per SM
```

Metric	Question answered
Occupancy	How much warp state is resident?
Utilization	How busy is a hardware unit?
Efficiency	How much issued work is useful?

6. Common Interview Questions

1. **Define occupancy.** Resident active warps divided by maximum supported warps per SM. Expected: ratio and SM scope. Mistake: percentage of busy cores.
2. **Does 100% occupancy mean best speed?** No; bandwidth, dependencies, divergence, and instruction mix still matter. Mistake: using it as the final objective.
3. **What limits occupancy?** Threads/warps, registers, shared memory, and resident-block limits. Mistake: naming block size alone.
4. **Why do registers affect it?** A finite SM register file must allocate registers for every resident thread. Mistake: assuming registers are allocated globally.
5. **Why can lower occupancy be faster?** More registers or shared memory per block can enable reuse and ILP. Mistake: forcing occupancy without checking spills.
6. **What is achieved occupancy?** Runtime average of active resident warps relative to maximum. Mistake: confusing it with eligible warps per cycle.
7. **Can tiny blocks hurt occupancy?** Yes; block-slot limits may be reached before warp capacity. Mistake: assuming more small blocks always fill the SM.
8. **How does grid size matter?** Too few blocks cannot populate all SMs; final waves create a tail. Mistake: considering only per-SM resource math.
9. **How do you tune occupancy?** Try sensible block sizes and resource variants, profile stalls and time. Mistake: blindly using the calculator's highest number.
10. **What occupancy hides memory latency?** No universal percentage; it depends on latency, ILP, instruction mix, and scheduler architecture. Mistake: quoting a fixed threshold.

7. Deep-Dive Questions

1. **Why is occupancy quantized?** Blocks, warps, registers, and shared memory allocate in discrete units, so a one-register change can cross a residency boundary.
2. **How does ILP substitute for occupancy?** Independent instructions from one thread/warp keep pipelines busy while earlier operations wait, reducing the number of warps required.
3. **What is a launch-bound annotation's tradeoff?** It can encourage compiler register choices that meet residency targets, but excessive pressure may spill to local memory.
4. **Why can synchronization make nominal occupancy misleading?** Many resident warps may be blocked at a barrier, leaving few eligible to issue.
5. **How does cache capacity interact with occupancy?** More concurrent warps can enlarge the working set and reduce cache locality, sometimes making higher occupancy slower.

8. Comparison Tables

High occupancy	Low occupancy
More resident warps	Fewer resident warps
More latency-hiding candidates	May rely on ILP/reuse
Less resource budget per thread/block	More registers/shared memory possible
Can increase cache contention	Can leave pipelines idle
Theoretical	Achieved
Calculated from static resource limits	Measured during execution
Upper-bound planning tool	Includes launch waves and runtime behavior

9. Common Mistakes

- Equating occupancy with GPU utilization.
- Maximizing occupancy while causing register spills.
- Ignoring shared memory and block-slot limits.
- Using a block size not aligned to warp size.
- Testing only one input size or overlooking the grid tail.

10. Edge Cases / Special Cases

Persistent kernels intentionally use limited resident blocks; very small grids cannot fill the device; cooperative launches impose residency constraints; architecture-specific allocation granularities create cliffs; kernels limited by instruction dependencies may not benefit from added warps.

11. How to Explain in Interview

"Occupancy is resident warps divided by the SM's maximum resident warps. It provides warps for latency hiding, but it is a constraint metric, not a speed score. I use enough occupancy to remove scheduler starvation, then balance registers, shared-memory reuse, ILP, and measured runtime."

12. Quick Revision Notes

- Resident does not mean currently issuing.
- Limited by the minimum of several SM resources.
- Whole-block and allocation granularity cause step changes.
- 100% occupancy is not a goal by itself.
- Trap: reducing registers may spill and become slower.

13. Practice Tasks

1. Compute occupancy from a device's warp, block, register, and shared-memory limits.
2. Compile a kernel with several register caps and record spills, occupancy, and time.
3. Sweep block sizes from 64 to 1024 and explain occupancy cliffs.
4. Compare a small grid with a grid large enough for many waves.

14. Final Cheat Sheet

Item	Remember
Core	Resident warps / maximum warps per SM
Why	Supplies alternatives when warps stall
Most asked	Is higher always faster? No
Compare	Occupancy vs utilization vs efficiency
One line	Occupancy is latency-hiding capacity, not performance itself.

Block-Size Selection

1. Overview

Block-size selection chooses the number and shape of threads launched together as a block. A block is the unit assigned to an SM, shares on-chip memory, and synchronizes with block barriers. The choice affects warp packing, occupancy, resource allocation, memory mapping, and load balance. It is used in every GPU launch; interviewers ask it because “always use 256” is a useful starting heuristic, not a complete answer.

2. Core Idea

A block is like a work crew: too small and scheduling slots or shared resources are underused; too large and only one crew may fit, reducing flexibility. Start with a warp multiple such as 128 or 256, ensure the grid has many blocks, then benchmark nearby values.

For $N=1000$, a 256-thread 1D block launches four blocks and guards $i < N$. Threads are grouped into eight full warps per block. A 250-thread block still consumes eight warps, leaving six lanes unused in the last warp.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
Warp alignment	Multiples of warp size avoid partial internal warps.	128, 256 threads.	Boundary partial warp is usually unavoidable and small.
Block residency	Whole blocks must fit on an SM.	1024-thread block may allow only one block.	More threads is not automatically better.
Grid parallelism	Enough blocks are needed across all SMs.	Several waves of blocks.	Small grid cannot saturate a large GPU.
Dimensional shape	Map x/y/z threads to data locality.	16×16 image tile.	Linear lane order and coalescing matter.
Shared-memory tile	Block shape controls halo/reuse cost.	32×8 vs 16×16 stencil tile.	Compare reuse and boundary overhead.
Autotuning	Empirically choose among valid candidates.	Benchmark 128/256/512.	Hardware and kernel resources vary.

4. Real-World Example

A 2D convolution might use a 16×16 block. Threads cooperatively load a tile plus halo into shared memory, synchronize, and compute outputs. A 32×8 tile has the same 256 threads but different coalescing, halo-to-interior ratio, and shared-memory bank behavior; shape can matter as much as count.

5. Diagrams / Mental Models

```
grid -> blocks distributed to SMs -> blocks split into warps
256-thread block = 8 warps
250-thread block = 8 allocated warps, last has 26 useful lanes
```

Selection checklist: correctness limits → warp multiple → resource residency → data mapping → benchmark.

6. Common Interview Questions

- What is a good starting block size?** Usually 128–256 threads, warp-aligned; then measure. Expected: heuristic, not law. Mistake: one universal optimum.
- Why use a warp multiple?** It avoids a partially populated warp inside every block. Mistake: saying nonmultiples are invalid.
- Why not always 1024?** It can reduce resident blocks, scheduling flexibility, and resource availability. Mistake: equating block size with parallelism.
- Can 128 beat 256?** Yes, through better residency, balance, or resource fit. Mistake: assuming larger wins.
- How does block shape affect coalescing?** Consecutive lane IDs should map to adjacent memory addresses, usually along x. Mistake: considering only total threads.

6. **How does shared memory constrain the choice?** Per-block allocation limits blocks resident per SM. Mistake: ignoring dynamic shared memory.
7. **What if N is not divisible by block size?** Round grid size up and bounds-check. Mistake: dropping elements or requiring exact division.
8. **What is an occupancy API/calculator for?** It filters or suggests launch shapes based on resource limits. Mistake: treating its suggestion as benchmark proof.
9. **Why need many blocks?** Independent blocks distribute over SMs and provide multiple scheduling waves. Mistake: launching one huge block.
10. **When is a fixed block size justified?** After measurements on supported architectures or when algorithmic tile constraints dictate it. Mistake: hard-coding without documenting constraints.

7. Deep-Dive Questions

1. **Why can two equal-sized 2D blocks differ?** Lane-to-address mapping, halo surface area, bank conflicts, and boundary behavior differ.
2. **How does tail effect influence selection?** If the final wave has few blocks, SMs go idle; more, smaller blocks can improve distribution.
3. **When should one warp equal one block?** Warp-specialized tasks with warp primitives and little shared state; block-slot limits can still reduce occupancy.
4. **How do cooperative groups constrain blocks?** Required synchronization scope and cooperative-launch residency may impose exact shapes or grid limits.
5. **Can launch overhead favor larger blocks?** Blocks have scheduling and shared setup costs, but kernel launch overhead is primarily per launch, not per thread.

8. Comparison Tables

Small blocks		Large blocks	
Better distribution and residency flexibility		More cooperation within a block	
May hit block-slot limit		May permit few blocks per SM	
Less shared state per block		Can amortize tile/halo work	
1D	2D/3D		
Natural for vectors	Natural for images/volumes		
Simple address mapping	Shape influences locality and halo		

9. Common Mistakes

- Assuming 256 is always optimal.
- Using non-warp multiples without reason.
- Ignoring block shape and lane ordering.
- Maximizing occupancy instead of runtime.
- Benchmarking only warm-cache or tiny inputs.

10. Edge Cases / Special Cases

Algorithms may require a specific power of two; partial final blocks need safe barriers (do not return before a barrier reached by peers); dynamic shared memory changes residency at launch; very small inputs may be overhead-dominated; persistent kernels deliberately launch few blocks.

11. How to Explain in Interview

"I start with 128 or 256 warp-aligned threads, map lanes to contiguous memory, check register/shared-memory residency and grid size, then benchmark nearby sizes. Tile shape and useful work matter more than maximizing thread count."

12. Quick Revision Notes

- Block is scheduling, cooperation, and resource-allocation unit.
- Warp-aligned is preferred, not required.
- Count and shape both matter.
- Enough grid blocks are required for device-wide load balance.
- Trap: largest legal block is rarely automatically best.

13. Practice Tasks

1. Sweep vector-add blocks through 32–1024 and plot time.
2. Compare 16×16 , 32×8 , and 8×32 image blocks.
3. Add shared-memory tiling and recompute residency.
4. Test non-divisible sizes with a correctness checker.

14. Final Cheat Sheet

Item	Remember
Core	Choose threads/shape per cooperative scheduling unit
Why	Affects packing, locality, residency, balance
Asked	Why not always maximum size? Resource and flexibility costs
Compare	Small vs large; 1D vs multidimensional
One line	Start warp-aligned, map memory well, check resources, benchmark.

Register Pressure

1. Overview

Register pressure is the demand a kernel places on the finite per-SM register file, usually expressed as registers per thread plus lifetime overlap. Registers are the fastest thread-private storage, but high use can reduce occupancy; forcing use too low can spill values to local memory, which resides in device memory and is far slower. It matters in unrolled loops, large expressions, private arrays, and heavily tiled kernels. Interviewers test whether you understand this three-way tradeoff: registers, occupancy, and spills.

2. Core Idea

Registers are desks in a shared office. Giving each worker a large desk makes individual work quick, but fewer workers fit. Giving tiny desks admits more workers, but they repeatedly visit a distant storeroom.

The compiler performs liveness analysis and register allocation. More simultaneously live values require more registers. At launch, registers are reserved for all threads in resident blocks, in allocation units. If demand crosses a threshold, one fewer block may fit. If capped aggressively, excess values spill to local memory.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Liveness	Values needed later must remain stored.	Many accumulators across a loop.	Shorten live ranges before arbitrary caps.
Spilling	Compiler stores register values to local memory.	Stack-like load/store instructions.	"Local" is private address space, not on-chip guarantee.
Occupancy cliff	Small register increase can remove a resident block.	64→65 registers crosses allocation boundary.	Resource allocation is quantized.
ILP	Multiple independent accumulators increase register use and throughput.	Four-way reduction.	Lower occupancy can be worthwhile.
Unrolling/inlining	Removes overhead but expands live state/code.	Fully unrolled loop.	Compiler choices have tradeoffs.
Register cap	Compiler option/launch bound limits allocation.	<code>maxrregcount</code> .	Can trade occupancy for spills badly.

4. Real-World Example

A matrix-multiplication thread holds an output tile in registers. More accumulators increase reuse and independent fused multiply-adds, but fewer warps fit. Production kernels search tile sizes so the register tile is large enough for reuse without unacceptable spills or scheduler starvation.

5. Diagrams / Mental Models

```

more registers/thread
  -> more fast private state and possible ILP
  -> fewer resident threads/warps

hard cap too low -> spills -> local-memory traffic -> long latency

```

Optimize runtime, not any one arrow.

6. Common Interview Questions

1. **What is register pressure?** Demand for a limited register file from live per-thread values. Mistake: defining it only as register count.
2. **Why can it lower occupancy?** Every resident thread needs its allocation from the SM register file. Mistake: thinking blocks share each register.
3. **What is a spill?** A value is placed in local memory because registers are insufficient. Mistake: assuming local memory is fast shared memory.
4. **Should you always reduce registers?** No; recomputation/spills can exceed occupancy benefit. Mistake: optimizing compiler statistics alone.
5. **How do you observe pressure?** Compiler resource reports plus profiler spill, occupancy, and stall metrics. Mistake: guessing from variable count.
6. **Do source variables map one-to-one to registers?** No; allocation, reuse, optimization, and type width change mapping. Mistake: counting declarations.
7. **How can code reduce pressure?** Shorten live ranges, limit unnecessary unrolling/inlining, reuse temporaries, or change tile size. Mistake: moving everything to shared memory.
8. **Why do private arrays often spill?** Dynamic indexing prevents easy scalar register allocation. Mistake: assuming every local scalar/array stays in registers.
9. **Can higher pressure improve speed?** Yes through ILP, reuse, and fewer memory loads. Mistake: treating pressure as inherently bad.
10. **What happens at a register cliff?** Allocation granularity or total capacity reduces resident blocks abruptly. Mistake: expecting smooth occupancy changes.

7. Deep-Dive Questions

1. **Why may recomputation beat spilling?** A few arithmetic instructions can be cheaper than a high-latency memory round trip.
2. **How do register dependencies affect throughput?** Even with available registers, a long dependency chain stalls; multiple accumulators create ILP at the cost of pressure.
3. **Can spills hit cache?** Yes, but they still add instructions, consume cache/bandwidth, and are not predictably cheap.
4. **How does calling affect pressure?** Inlining may expand live ranges; non-inlined calls may require stack/local storage and inhibit optimization.
5. **Why is architecture portability hard?** Register-file size, allocation units, instruction set, and occupancy limits differ.

8. Comparison Tables

Registers	Shared memory	Local memory
Thread-private, fastest	Block-shared, explicit	Thread-private address space, device-backed
Limited by register file	Limited per SM/block	High latency; cached architecture-dependently
Natural allocation	Forced low register cap	
Compiler balances state	May raise occupancy	
Could limit residency	May introduce spills/recomputation	

9. Common Mistakes

- Calling local memory on-chip scratchpad.
- Applying a register cap without checking spills.
- Disabling useful unrolling everywhere.
- Counting C++ variables as physical registers.
- Ignoring 64-bit values and allocation granularity.

10. Edge Cases / Special Cases

Compiler optimizations can remove variables entirely; constant-index private arrays may be scalarized; debug builds distort allocation; one extra live value can trigger a residency cliff; spills may appear acceptable for one input because caches hide them.

11. How to Explain in Interview

“Register pressure is demand from simultaneously live per-thread values on the SM’s finite register file. More registers can improve reuse and ILP but reduce occupancy; too few cause spills to local memory. I inspect compiler/profiler data and optimize total runtime, not register count.”

12. Quick Revision Notes

- Registers are per thread but physically finite per SM.
- Liveness, unrolling, inlining, and tile size drive demand.
- Spills are memory operations.
- Allocation cliffs make behavior non-linear.
- Trap: lower register count can be slower.

13. Practice Tasks

1. Compile an unrolled reduction and inspect registers/spills.
2. Sweep an unroll factor and graph registers, occupancy, and time.
3. Shorten a temporary’s live range and compare generated code.
4. Force several register caps and identify the first spill cliff.

14. Final Cheat Sheet

Item	Remember
Core	Live-value demand on finite per-SM registers
Why	Trades fast state/ILP against occupancy/spills
Asked	Why not minimize registers? Spills and lost reuse
Compare	Register vs shared vs local memory
One line	Use enough registers to avoid expensive memory, but not so many that residency starves.

Instruction Throughput

1. Overview

Instruction throughput is the sustainable rate at which a GPU issues or completes a class of instructions, often stated per cycle per SM or across the device. It differs from latency: throughput asks how many operations can start/finish over time; latency asks how long one dependent operation takes. It matters in compute-heavy kernels, reductions, transcendentals, address generation, and mixed integer/floating-point code. Interviewers ask it to see whether you can use hardware peak numbers without confusing operations, instructions, lanes, and dependencies.

2. Core Idea

A pipelined car wash may take five minutes per car (latency) yet finish one car every minute (throughput). GPUs reach high throughput by overlapping many independent operations across lanes and warps.

For $c = a*b + c$, an FMA is one instruction but often counted as two floating-point operations. Peak FLOP/s derives from active units $\times$ operations per instruction $\times$ clock, but real throughput is reduced by dependencies, issue limits, instruction mix, stalls, and insufficient work.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Latency vs throughput	Time per dependent result vs sustained rate.	Pipeline takes 4 cycles, accepts 1/cycle.	Most common distinction.
Issue vs execution	Schedulers dispatch; pipelines execute.	Eligible warp needs correct unit.	Bottleneck may be front end or unit.
Instruction mix	FP, integer, load/store, special functions use different resources.	Address math competes differently from FMA.	Peak FP rate may be irrelevant.
Dependency chains	Next operation waits for previous result.	Serial sum.	Use ILP or parallel reduction.
Dual issue/concurrency	Some architectures overlap compatible work.	Integer address plus FP math.	Architecture-specific; avoid universal claims.
FLOPs vs instructions	One instruction can perform many lane operations.	Warp FMA = 32 instructions-lane results, 64 FLOPs.	State counting convention.

4. Real-World Example

A neural-network convolution maps to tensor-core matrix operations. Its theoretical throughput is enormous only when shapes, alignment, precision, occupancy, and data supply suit those units. Small matrices or layout conversions can leave tensor pipelines underfed.

5. Diagrams / Mental Models

```

ready warps -> scheduler/issue -> execution pipeline -> result
                ^ issue limit           ^ unit throughput
dependencies/readiness can stop flow before either peak is reached

```

6. Common Interview Questions

1. **Throughput vs latency?** Throughput is results per time; latency is time for one dependency. Mistake: using them interchangeably.
2. **What is peak FLOP/s?** Hardware's ideal arithmetic rate under specified precision/operations. Mistake: promising application performance.
3. **Why count FMA as two FLOPs?** It performs a multiply and add mathematically. Mistake: calling it two instructions.
4. **Why might a compute kernel miss peak?** Dependencies, instruction mix, issue limits, divergence, or inadequate data/work. Mistake: blaming memory only.
5. **What is instruction-level parallelism?** Independent instructions from a thread/warp that overlap. Mistake: confusing it with more threads.
6. **Can high occupancy guarantee throughput?** No; resident warps may be stalled or target a saturated unit. Mistake: occupancy equals issue rate.
7. **What limits transcendental functions?** Special-function-unit throughput/latency and approximations. Mistake: treating `sin` like add.
8. **How do you measure throughput?** Profile executed instructions/unit utilization and normalize by cycles/time. Mistake: use source operation count alone.
9. **What is a dependency chain?** Each instruction consumes the previous result, preventing overlap. Mistake: assuming many loop iterations are independent.
10. **Why does precision matter?** FP64, FP32, FP16, integer, and tensor paths have different rates. Mistake: quote one "GPU FLOPS" value.

7. Deep-Dive Questions

1. **When can integer work bottleneck FP code?** Address calculation, bounds logic, and indexing may saturate integer/issue resources while FP units wait.
2. **Why can unrolling help?** It exposes independent operations and reduces control overhead, but raises code size/register pressure.
3. **What is Little's Law intuition here?** Concurrency required $\approx$ throughput $\times$ latency; enough in-flight independent operations fill the pipeline.
4. **Why is SASS more reliable than source for counting?** Compiler fusion, elimination, vectorization, and lowering change actual instructions.
5. **Can lower instruction count be slower?** Yes if replacements use a low-throughput unit, create dependencies, or reduce occupancy.

8. Comparison Tables

Latency	Throughput
Cycles until one result is usable	Operations/instructions per cycle

Latency	Throughput
Hurts dependency chains	Limits sustained independent work
Hidden with overlap	Reached by keeping pipeline supplied
FLOP count	Instruction count
Algorithmic operations	Machine operations issued
FMA commonly counts as 2	FMA is one instruction

9. Common Mistakes

- Comparing measured FLOP/s with the wrong precision peak.
- Ignoring integer, conversion, and special-function instructions.
- Treating latency tables as throughput tables.
- Counting optimized-away source operations.
- Assuming fewer instructions means faster execution.

10. Edge Cases / Special Cases

Tensor/specialized units require supported shapes and types; boost clocks and power limits change peaks; denormal/precision modes may alter behavior; compiler fusion changes counts; tiny kernels cannot amortize startup or fill pipelines.

11. How to Explain in Interview

"Instruction throughput is the sustained instruction or operation rate, distinct from the latency of one dependent instruction. GPUs approach it with many lanes, ready warps, and ILP, but the actual limit depends on issue width, execution-unit mix, dependencies, and data supply."

12. Quick Revision Notes

- Throughput = rate; latency = delay.
- FMA: one instruction, conventionally two FLOPs.
- Dependencies reduce achievable rate.
- Different types use different pipelines/rates.
- Trap: theoretical peak is a roof, not a prediction.

13. Practice Tasks

1. Benchmark independent accumulators versus one dependent accumulator.
2. Compare add, FMA, divide, and transcendental loops.
3. Inspect generated machine instructions and recount operations.
4. Use profiler unit-utilization metrics to identify the saturated pipeline.

14. Final Cheat Sheet

Item	Remember
Core	Sustainable instruction/operation rate
Why	Sets compute-side performance ceiling
Asked	Throughput vs latency; why peak is missed
Compare	FLOPs vs instructions; issue vs execution
One line	Throughput is how fast the pipeline is fed and drained, not how long one operation waits.

Latency Hiding

1. Overview

Latency hiding keeps GPU pipelines productive by executing other ready work while an instruction waits, especially for memory. It does not shorten the delayed operation. GPUs use warp-level parallelism (switch warps), instruction-level parallelism (issue independent operations), asynchronous copies, and pipelining. It matters in almost every kernel and is asked to test why GPUs support thousands of lightweight threads.

2. Core Idea

When one restaurant order waits in an oven, the cook prepares another. Likewise, after warp A issues a long-latency load, a scheduler can issue warp B. If A also has independent arithmetic, that work can overlap its own wait. The sequence is: issue load → mark dependent instruction unready → choose another eligible warp/instruction → return later when data arrives. Hiding succeeds only if enough independent useful work exists.

3. Important Subtopics

Subtopic	Meaning / importance	Example	Interview angle
TLP	Many ready warps cover stalls.	Switch to another resident warp.	Occupancy supplies candidates, not readiness.
ILP	Independent instructions overlap within a warp.	Multiple accumulators.	Costs registers.
Memory-level parallelism	Multiple outstanding requests overlap.	Load several elements before use.	Bandwidth and queue limits apply.
Software pipelining	Overlap stages across iterations/tiles.	Load tile $k+1$ while computing k .	Requires buffers and synchronization.
Eligibility	Resident warp whose next instruction can issue.	Warp not blocked at barrier.	Resident $\neq$ eligible.
Stall analysis	Profiler identifies why no warp issued.	Long scoreboard wait.	Diagnose before raising occupancy.

4. Real-World Example

Tiled matrix multiplication double-buffers shared-memory tiles. While computation consumes tile k , asynchronous operations fetch tile $k+1$; a synchronization point protects reuse. This overlaps global-memory latency with useful FMA work when enough computation exists per tile.

5. Diagrams / Mental Models

```
time ->
warp A: LOAD -----wait----- USE
warp B:      COMPUTE  COMPUTE
warp C:      LOAD -----
SM:      useful issue fills A's waiting interval
```

Latency is hidden, not removed; insufficient eligible work exposes the gap.

6. Common Interview Questions

1. **What is latency hiding?** Scheduling independent work during a wait. Expected: latency remains. Mistake: saying memory gets faster.
2. **Why do GPUs need many threads?** Cheap resident warps provide alternatives during stalls. Mistake: every thread runs simultaneously.
3. **How does occupancy help?** It increases potential resident warps. Mistake: guarantees eligible warps.
4. **What is ILP?** Independent operations from one instruction stream overlap. Mistake: equating it with TLP.
5. **How can ILP hurt?** More live values raise register pressure and lower occupancy. Mistake: unlimited unrolling.
6. **What is a scoreboard stall?** A needed operand from a prior instruction/load is not ready. Mistake: a thread-synchronization primitive.
7. **Can bandwidth saturation coexist with hidden latency?** Yes; latency can be covered while throughput is bandwidth-limited. Mistake: treating them as opposites.
8. **Why do barriers reduce hiding?** Many warps can simultaneously become ineligible. Mistake: resident warps always execute.
9. **How does prefetching help?** It starts requests early, creating overlap. Mistake: prefetching unlimited data without capacity cost.
10. **How do you know latency is exposed?** Profiler shows low eligible warps/issue and dependency or memory stalls. Mistake: conclude from high memory latency alone.

7. Deep-Dive Questions

1. **How much concurrency is enough?** Roughly latency $\times$ desired throughput, adjusted for instruction mix and scheduler structure; profile rather than use one magic occupancy.
2. **Why can pointer chasing resist hiding?** Each address depends on the prior load, limiting ILP and memory-level parallelism.
3. **How does double buffering trade resources?** It overlaps stages but doubles buffer state and can reduce occupancy.

4. **Can caches eliminate the need?** Caches reduce average latency/traffic but misses and dependencies still require overlap.
5. **Why may more warps stop helping?** Bandwidth or execution units saturate, or added working sets harm cache locality.

8. Comparison Tables

TLP	ILP
Switch among warps	Overlap instructions within a warp/thread
Uses occupancy	Uses independent operations/registers
Robust for irregular stalls	Useful when few warps reside
Reduce latency	Hide latency
Cache/local memory improves the operation	Other work covers its wait

9. Common Mistakes

- Saying hidden latency disappeared.
- Maximizing occupancy without checking eligible warps.
- Prefetching so far ahead that registers spill.
- Ignoring barriers and dependency chains.
- Confusing latency saturation with bandwidth saturation.

10. Edge Cases / Special Cases

Pointer chains expose latency; small grids offer little TLP; synchronized phases can align stalls; persistent kernels may hide with ILP or staged queues; asynchronous operations require correct lifetime and completion synchronization.

11. How to Explain in Interview

"Latency hiding schedules other ready warps or independent instructions while an operation waits. Occupancy helps supply candidates, ILP and prefetching create overlap, but dependencies, barriers, and resource pressure determine whether latency is actually hidden."

12. Quick Revision Notes

- Hidden $\neq$ reduced.
- TLP switches warps; ILP overlaps independent instructions.
- Resident is not necessarily eligible.
- More concurrency stops helping after another roof is reached.
- Trap: 100% occupancy does not prove latency is hidden.

13. Practice Tasks

1. Compare dependent pointer chasing with independent array loads.
2. Add multiple accumulators and measure register/latency tradeoffs.

3. Build single- and double-buffered tile loops.
4. Correlate eligible-warps and stall metrics with runtime.

14. Final Cheat Sheet

Item	Remember
Core	Cover a wait with independent useful work
Why	Keeps pipelines issuing
Most asked	Hiding vs reducing latency
Compare	TLP vs ILP; latency vs bandwidth
One line	GPUs tolerate long latency by having something else ready to run.

Arithmetic Intensity

1. Overview

Arithmetic intensity (AI) is useful arithmetic operations divided by bytes transferred at a specified memory boundary, commonly FLOPs per byte from device memory. It predicts whether performance is more likely constrained by compute throughput or memory bandwidth through the Roofline model. It is used in algorithm selection, tiling, fusion, and precision decisions. Interviewers ask it to connect code, data movement, and hardware limits quantitatively.

2. Core Idea

AI is “work obtained per delivery.” If vector addition reads **a** and **b** and writes **c**, it performs one add while moving roughly 12 bytes for FP32, so $AI \approx 1/12$ FLOP/byte (ignoring cache/write-policy details). Reusing a matrix tile from shared memory lets many FMAs reuse each global byte, raising device-memory AI.

Compare AI with the machine balance point:

```
ridge point = peak compute (FLOP/s) / peak bandwidth (byte/s)
attainable performance <= min(peak compute, AI × bandwidth)
```

3. Important Subtopics

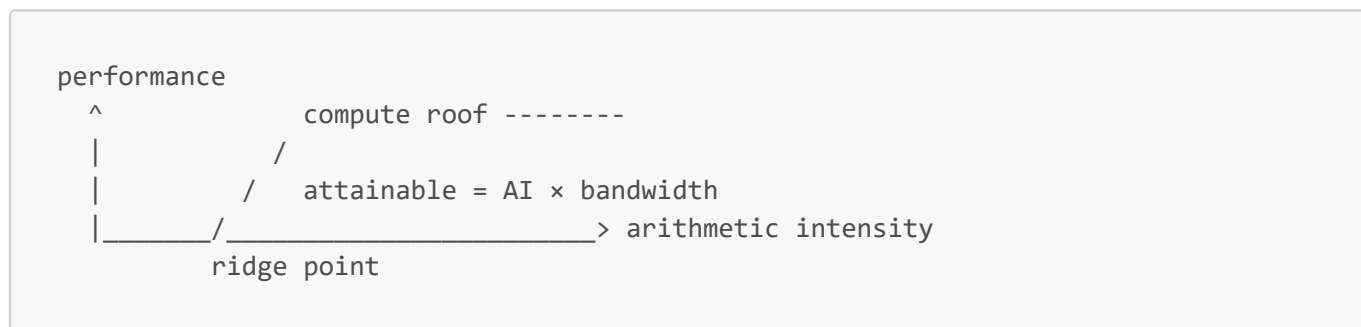
Subtopic	Meaning / importance	Example	Interview angle
Boundary choice	Bytes differ at DRAM, cache, shared memory.	Cache hit avoids DRAM bytes.	Always state the level.
Useful vs executed FLOPs	Algorithmic work may differ from machine work.	Redundant recomputation.	Roofline convention must be clear.

Subtopic	Meaning / importance	Example	Interview angle
Reuse	More operations per fetched value raise AI.	Tiled GEMM.	Tiling changes traffic, not required math.
Fusion	Keeps intermediates on-chip.	Bias+activation after convolution.	Can increase registers and reduce reuse elsewhere.
Precision	Smaller values reduce bytes and change compute peak.	FP16 vs FP32.	Recalculate both roofs.
Ridge point	AI at which bandwidth and compute ceilings meet.	peak FLOP/s ÷ GB/s.	Hardware-specific classification.

4. Real-World Example

Naive matrix multiplication repeatedly fetches operands, while tiled GEMM loads submatrices once and reuses them across many FMAs. The mathematical FLOP count stays near **2MNK**; device-memory bytes fall, AI rises, and the kernel can move from a bandwidth roof toward the compute roof.

5. Diagrams / Mental Models



6. Common Interview Questions

1. **Define AI.** Useful operations divided by bytes transferred at a named hierarchy level. Mistake: operations per second.
2. **Units?** Usually FLOP/byte. Mistake: FLOP/s.
3. **Why does it matter?** It links an algorithm's data reuse to bandwidth/compute roofs. Mistake: calling it direct runtime.
4. **What is the ridge point?** Peak compute divided by peak bandwidth. Mistake: universal constant.
5. **How can tiling raise AI?** Reuse fetched data on-chip for multiple operations. Mistake: claiming tiling adds necessary FLOPs.
6. **Does more arithmetic always raise useful AI?** No; redundant work inflates executed operations without algorithmic value. Mistake: add dummy math.
7. **How does fusion help?** Avoids writing/reading intermediates. Mistake: fusion always wins despite pressure/locality.
8. **Is AI hardware-independent?** Algorithmic AI at a defined boundary can be, but realized traffic and ridge classification depend on hardware/cache. Mistake: omit boundary.
9. **Calculate vector-add AI.** About 1 FLOP/12 DRAM bytes for FP32 under simple traffic assumptions. Mistake: ignore output store.

10. **Does high AI guarantee compute-bound behavior?** No; dependencies, poor utilization, or another pipeline may limit it. Mistake: Roofline as proof rather than bound.

7. Deep-Dive Questions

1. **What is hierarchical Roofline?** Separate AI and bandwidth ceilings for DRAM, caches, and on-chip memories to locate the limiting level.
2. **How do cache write policies affect byte counts?** Write allocation, eviction, and transaction granularity can add traffic beyond source-level bytes.
3. **Can recomputation be beneficial?** Yes if cheap arithmetic replaces costly data movement, despite more executed FLOPs.
4. **Why might fusion lower performance?** Larger kernels can increase registers, reduce occupancy, and lose specialized library paths.
5. **How do sparse kernels complicate AI?** Metadata, irregular transactions, low lane utilization, and variable useful operations alter both numerator and denominator.

8. Comparison Tables

Low AI	High AI
Little work per byte	Much reuse/work per byte
Often bandwidth-sensitive	Often compute-sensitive
Example: vector copy/add	Example: well-tiled GEMM

Algorithmic AI	Measured AI
Ideal useful work / modeled bytes	Counters reveal actual traffic/executed work
Good design estimate	Includes cache/transaction effects

9. Common Mistakes

- Omitting stores, metadata, or transaction overhead.
- Mixing decimal GB with binary GiB silently.
- Failing to state memory boundary.
- Adding useless operations to “improve” AI.
- Assuming AI alone identifies every bottleneck.

10. Edge Cases / Special Cases

Cache-resident inputs change effective DRAM AI; atomics include serialization effects beyond bytes; compression/sparsity changes useful and physical traffic; mixed precision changes operation rates; small problems may be launch/latency-bound instead.

11. How to Explain in Interview

“Arithmetic intensity is useful FLOPs per byte moved at a stated memory level. In Roofline, performance is bounded by $\min(\text{peak compute}, \text{AI} \times \text{bandwidth})$. I raise AI through reuse, tiling, or fusion—not artificial arithmetic—and validate actual traffic with profiling.”

12. Quick Revision Notes

- AI = operations / bytes, not operations / second.
- State the memory boundary and counting assumptions.
- Ridge = peak compute / peak bandwidth.
- Reuse raises AI by reducing transfers.
- Trap: high AI is not a universal guarantee of compute saturation.

13. Practice Tasks

1. Calculate AI for copy, SAXPY, reduction, and matrix multiply.
2. Compare naive and tiled matrix multiplication traffic.
3. Use profiler byte counters to compute measured DRAM AI.
4. Plot kernels on a Roofline chart for one GPU.

14. Final Cheat Sheet

Item	Remember
Core	Useful operations per byte transferred
Why	Relates data reuse to performance ceilings
Most asked	Vector-add AI; ridge point; how tiling helps
Compare	Low vs high AI; modeled vs measured
One line	AI tells how much useful computation each delivered byte enables.

Memory-Bound vs Compute-Bound Kernels

1. Overview

A **memory-bound kernel** is limited mainly by sustainable data movement; a **compute-bound kernel** is limited mainly by arithmetic or instruction execution capacity. "Bound" identifies the resource whose improvement would most improve runtime under the current implementation and input. This classification guides optimization: coalescing, reuse, and traffic reduction for memory limits; instruction efficiency, specialized units, and ILP for compute limits. Interviewers expect evidence, not a guess based on kernel name.

2. Core Idea

A factory can be starved by slow material delivery or limited by machine processing. Roofline gives a first test: if $AI \times \text{bandwidth} < \text{peak compute}$, the bandwidth roof is lower; otherwise the compute roof is lower. Then profile achieved bandwidth, unit utilization, stalls, and instruction mix because kernels can instead be latency-, launch-, synchronization-, or occupancy-bound.

Example: vector addition has low AI and is usually memory-bound; tiled GEMM has high AI and can be compute-bound. Poorly coalesced GEMM may still be memory-limited.

3. Important Subtopics

Subtopic	Meaning / importance	Example	Interview angle
Bandwidth-bound	Sustained bytes/s near practical ceiling.	Streaming copy.	Optimize traffic/coalescing.
Compute-throughput-bound	Execution pipeline near its rate ceiling.	Large tuned GEMM.	Use efficient instructions/units.
Latency-bound	Dependencies leave pipelines idle without saturating bandwidth.	Pointer chasing.	Not every memory stall is bandwidth-bound.
Roofline	Combines AI and hardware roofs.	Position relative to ridge.	Upper bound plus diagnosis.
Bottleneck migration	Optimizing one roof reveals another.	Tiling moves DRAM limit to compute.	Re-profile after changes.
Practical ceilings	Sustainable rates below marketing peaks.	ECC, clocks, workload mix.	Compare against measured baselines.

4. Real-World Example

A database-style GPU scan reads columns and evaluates a simple predicate, so bytes dominate. Compressing columns or fusing projection reduces traffic. Adding complicated expression evaluation raises AI and may move the same scan toward instruction or compute limits; classification belongs to a concrete implementation and workload.

5. Diagrams / Mental Models

```

low AI ----- ridge ----- high AI
bandwidth roof likely                compute roof likely

Evidence chain: model -> benchmark -> profiler -> change -> re-profile

```

6. Common Interview Questions

1. **Define memory-bound.** More performance is chiefly limited by data movement rate. Mistake: any kernel containing loads.
2. **Define compute-bound.** Execution throughput is the main limiting resource. Mistake: many arithmetic statements prove it.
3. **How do you classify a kernel?** Estimate AI/Roofline, then verify achieved bandwidth, unit utilization, and stalls. Mistake: use one metric.
4. **Is vector addition memory-bound?** Usually: about one FLOP per 12 FP32 bytes. Mistake: ignore cache/small-size cases.

5. **Is matrix multiplication compute-bound?** A well-tiled large GEMM often is; naive or small versions may not be. Mistake: classify by algorithm name.
6. **How optimize memory-bound code?** Reduce bytes, coalesce, reuse/cache, fuse passes, and use suitable precision/layout. Mistake: merely add threads after saturation.
7. **How optimize compute-bound code?** Reduce costly instructions, expose ILP, use FMA/tensor units and suitable precision. Mistake: optimize DRAM traffic already below relevance.
8. **Can a kernel be both?** At the ridge, ceilings may be comparable; different phases/levels can have different limits. Mistake: demand one permanent label.
9. **Why is low bandwidth not proof of compute-bound?** Latency, poor coalescing, dependencies, or insufficient parallelism may prevent saturation. Mistake: binary inference from utilization.
10. **What if neither resource is saturated?** Investigate latency, launch overhead, synchronization, divergence, occupancy, and instruction issue. Mistake: force Roofline's two labels.

7. Deep-Dive Questions

1. **How can you experimentally test a memory limit?** Vary bytes per element while holding useful compute similar, compare to a copy benchmark, and inspect traffic/coalescing counters.
2. **How can you test a compute limit?** Change arithmetic work or precision and see whether time tracks instruction demand while bandwidth remains below its roof.
3. **Why can cache-bound differ from DRAM-bound?** Reused data may avoid DRAM yet saturate an L1/L2/shared-memory path; hierarchical Roofline separates them.
4. **How does kernel fusion migrate bottlenecks?** It removes intermediate traffic but raises code size, registers, and arithmetic concentration.
5. **What does Amdahl's law add?** Optimizing the dominant kernel/resource only helps in proportion to its share of end-to-end time.

8. Comparison Tables

Memory-bound	Compute-bound
Low effective AI, bandwidth near ceiling	High AI, execution units near ceiling
Optimize bytes, locality, coalescing	Optimize instruction mix, ILP, specialized units
Extra arithmetic may be cheap	Extra memory optimization may have little effect
Bandwidth-bound	Latency-bound
Many requests saturate transfer rate	Too few independent requests expose wait time
More concurrency rarely helps after saturation	More TLP/ILP/prefetch may help

9. Common Mistakes

- Calling every load-stalled kernel bandwidth-bound.
- Using theoretical peaks instead of sustainable ceilings.
- Ignoring problem size, cache state, layout, and precision.
- Optimizing one bottleneck without re-profiling.
- Looking only at percent utilization without elapsed-time impact.

10. Edge Cases / Special Cases

Tiny kernels are often launch-bound; irregular gathers can be latency/transaction-bound without high bandwidth; atomic contention is serialization-bound; mixed phases need phase-level analysis; thermal/power throttling changes practical roofs; input distributions can change divergence and traffic.

11. How to Explain in Interview

"A memory-bound kernel is capped mainly by data movement, while a compute-bound kernel is capped by execution throughput. I estimate arithmetic intensity against the Roofline ridge, then confirm with achieved bandwidth, pipeline utilization, and stalls. If neither roof is approached, I investigate latency, divergence, synchronization, or launch overhead."

12. Quick Revision Notes

- Classification is implementation-, input-, and hardware-specific.
- Low AI suggests memory-bound; high AI suggests compute-bound.
- Suggestion is not proof: profile.
- Bandwidth-bound differs from memory-latency-bound.
- Trap: bottlenecks migrate after optimization.

13. Practice Tasks

1. Benchmark copy, vector add, reduction, naive GEMM, and tiled GEMM.
2. Compute modeled AI and compare with measured traffic.
3. Add arithmetic per loaded element and observe the ridge transition.
4. Optimize coalescing, then re-profile to find the next bottleneck.
5. Explain why a pointer-chasing kernel can show low bandwidth and still wait on memory.

14. Final Cheat Sheet

Item	Remember
Core	Dominant limit is data movement or execution throughput
Why	Determines which optimization can matter
Most asked	How to diagnose; vector add vs GEMM; bandwidth vs latency
Compare	Memory vs compute; bandwidth vs latency bound
One line	Use Roofline to hypothesize the bottleneck and profiler evidence to prove it.

Cross-Topic Interview Strategy

When given an unfamiliar kernel, reason in this order: verify correct/coalesced work mapping; estimate arithmetic intensity; identify the likely hardware roof; inspect divergence and instruction dependencies; check whether registers/shared memory/block size provide enough eligible warps; then profile and change one limiting factor at a time. A strong answer states assumptions and avoids universal launch sizes or occupancy targets.